DEVPILOT AI

Frontend Developer Documentation

Complete frontend implementation guide

Implementation guide for the approved DevPilot AI business flow: UI structure, page behavior, access rules, states and API integration points.

Document Version	1.0
Project	DevPilot AI
Audience	Frontend Developer / UI Engineer
Product promise	Debug Smarter. Learn Faster.

Contents

0. Source of Truth 5
Primary Source Documents 5
Important 5
1. Product Positioning 6
2. Users and Access 7
Role 1: Public Visitor 7
Role 2: Verified Developer 7
Role 3: Administrator 8
3. Global Frontend Rules 9
3.1 Responsive Scope 9
3.2 Global Navigation 9
3.3 Visual Identity 9
3.4 Typography 10
3.5 Buttons 10
3.6 Common Statuses 11
3.7 Standard UI States 11
3.8 Security UX 11
4. Information Architecture 13
5. Page Implementation Order 14
6. Page Specifications 15
6.1 Landing Page 15
6.2 Articles 16
6.3 Article Details 17
6.4 Login 19
6.5 Register 20
6.6 Email Verification 21
6.7 Forgot Password 21
6.8 Reset Password 22
6.9 Developer Dashboard 23
6.10 New Problem / Submit Problem 24
6.11 Problem Details 25
6.12 AI Debugging Workspace 26
6.13 Problem History 28
6.14 My Solutions 29
6.15 Knowledge Base – Developer View 29

6.16 Challenges 30
6.17 Challenge Details 31
6.18 Challenge Workspace 31
6.19 Challenge Result 32
6.20 GitHub Integration 33
6.21 GitHub Repositories 34
6.22 Repository Details 35
6.23 Repository Debugging 35
6.24 Code Review 36
6.25 Code Review Details 37
6.26 My Activity 38
6.27 Bookmarks 38
6.28 Notifications 39
6.29 Profile 40
6.30 Settings 40
7. Admin Page Specifications 42
7.1 Admin Dashboard 42
7.2 Users Management 42
7.3 Problems Management 43
7.4 Articles Management 43
7.5 Challenges Management 43
7.6 AI Providers 44
7.7 AI Models 44
7.8 AI Usage / Requests 44
7.9 System Monitoring 44
8. Core Shared Components 46
9. API Integration Map 47
Auth 47
User 47
Problems 47
Search 48
Articles 48
Challenges 48
GitHub 48
Code Review 48
Telegram 49
AI Gateway / Admin 49

10. Authentication Flow 50
11. Route Guards 51
12. AI UX Requirements 52
13. Knowledge Reuse UX 52
14. Article Evolution UX 53
15. GitHub UX Rules 54
16. File / Attachment UX 54
17. Error Handling 55
18. Loading and Performance 55
19. Accessibility 56
20. Data Display Rules 56
21. Search and Filter Rules 57
22. Dashboard Data Relationship 58
23. Frontend Security Checklist 59
24. UX Do's and Don'ts 60
26. Design System Implementation 61
27. Page-Level State Matrix 62
28. Implementation Notes for the Frontend Developer 64
Recommended Architecture 65
29. Final Business Acceptance Checklist 66
30. Final Frontend Principle 68

0. Source of Truth

The frontend must be built around the approved DevPilot AI business model. The backend and database are already designed around this workflow.

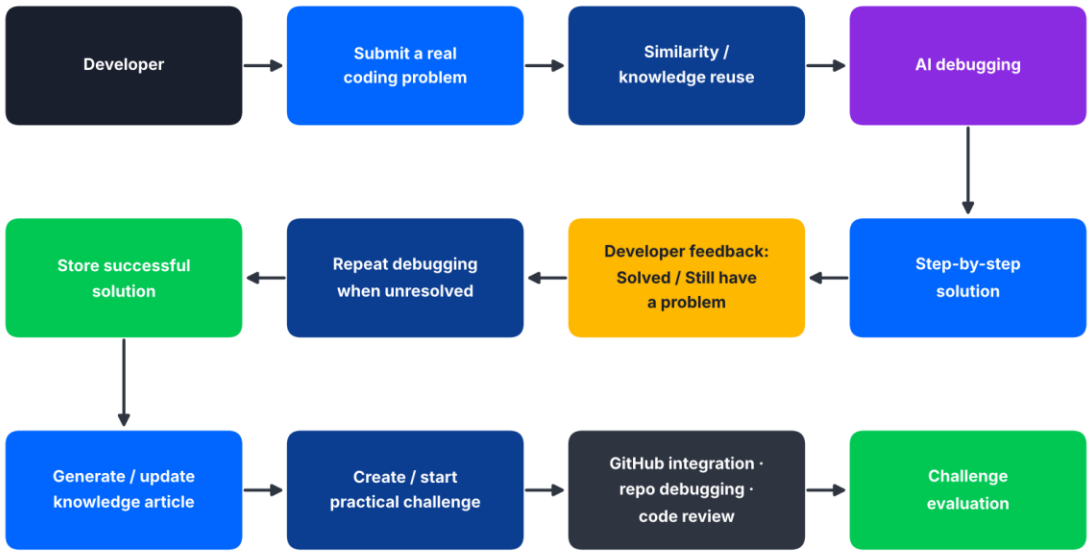


Figure 1 – Approved DevPilot AI business flow

Primary Source Documents

- DevPilot_AI_SRS.txt
- DevPilot_AI_API_Specification.txt
- DevPilot_AI_Database_Documentation_Final.txt
- DevPilot_AI_AI_Architecture_and_LLM_Routing.txt

Important

- Backend/API behavior defined in the source documents is authoritative.
- Frontend routes listed in this document are implementation recommendations unless an existing backend route has already been explicitly defined.
- Do not add product features merely because they look good in a UI.
- The product is developer-centered and problem-centered.
- The main product value is AI debugging + proven knowledge reuse + learning from solved problems + practical challenges + GitHub workflow.

1. Product Positioning

DevPilot AI is a developer-only platform. Core promise:

> **Core promise**
> "Debug Smarter. Learn Faster."

The platform should feel like	The interface must NOT feel like
A professional developer tool.	A generic social network.
An AI debugging workspace.	A generic blog.
A knowledge system built from real solved problems.	A generic AI chatbot.
A learning/practice layer connected to actual debugging work.	A generic online course platform.

The UI must continuously reinforce this product loop:



Figure 2 – Product loop to reinforce on every screen

2. Users and Access



Figure 3 – Role capability matrix

Role 1: Public Visitor

Can	Cannot
Open the landing page.	Use authenticated developer workspace.
Browse the public knowledge articles.	Start a challenge.
Search public knowledge where allowed.	Use private developer data.
Open article details.	View private debugging history.
View article problem, root cause, solution, best practices, prevention, statistics, related problems, and available challenge information.	Use private GitHub repositories.
Access public Telegram / Discord entry points when enabled.	Run authenticated debugging workflows.

Role 2: Verified Developer

Email verification is required before normal authenticated access.

- Access developer dashboard.
- Submit problems.
- Run AI debugging.
- Continue debugging after “Still Have a Problem”.
- View debugging history.

- View successful solutions.
- Reuse stored solutions.
- Browse the knowledge base.
- Start challenges.
- View challenge attempts/results.
- Connect GitHub.
- Debug repositories.
- Request AI code reviews.
- Manage profile/settings.
- View personal activity, bookmarks, and notifications.

Role 3: Administrator

- Access admin management screens.
- Manage users.
- Manage problems/knowledge.
- Manage challenges.
- Manage AI providers/models/usage.
- Monitor system status.
- Access administrator-only article/version information.

3. Global Frontend Rules

3.1 Responsive Scope

Approved visual planning focused on Desktop. The initial implementation should prioritize Desktop and large laptop/monitor widths. Responsive behavior is still required for usability. Do not create a separate mobile visual design unless explicitly requested.

3.2 Global Navigation

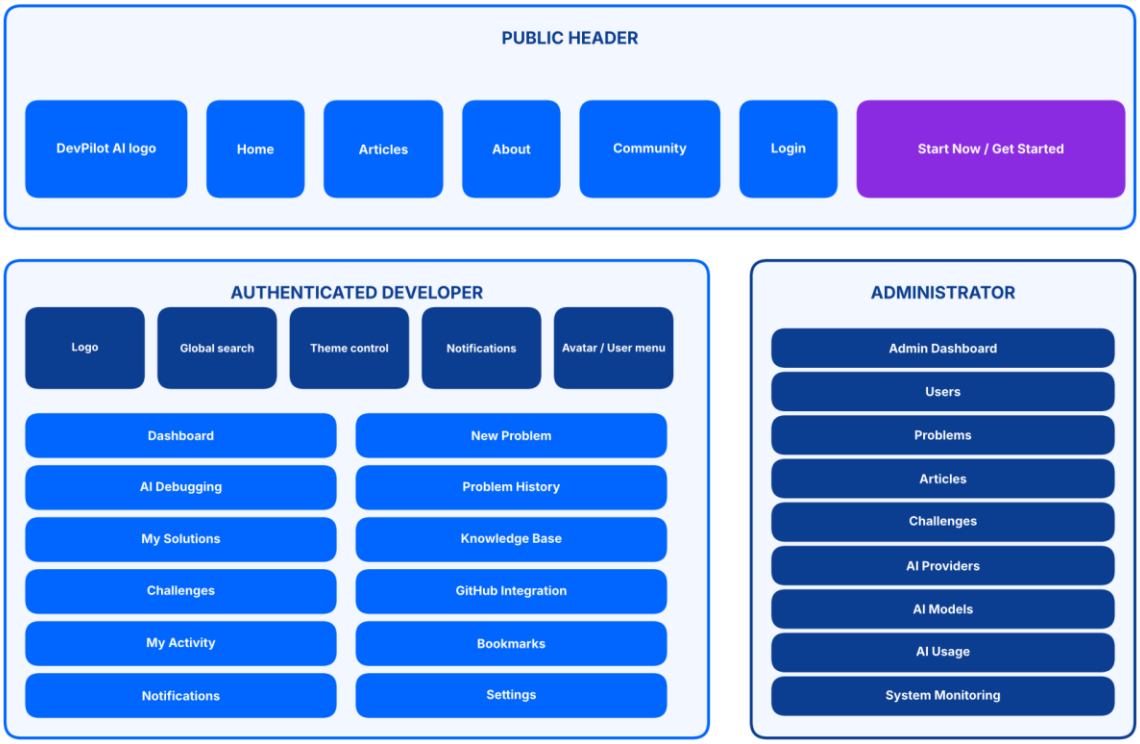


Figure 4 – Public header, authenticated developer header and sidebar, administrator navigation

3.3 Visual Identity

- Dark navy primary surfaces
- Electric / neon blue primary accent
- Purple / indigo secondary accent
- White / very light text
- Green for success
- Yellow / amber for warnings
- Red for errors
- Rounded cards
- Soft shadows
- Subtle glows
- Glass-like panels where appropriate

- Developer-oriented code blocks
- Clean modern sans-serif typography
- Consistent DevPilot AI geometric logo

> **Important**
> Do not replace the approved DevPilot AI logo with a different logo concept. Do not introduce unrelated colors or styles.

3.4 Typography

Role	Guidance
H1	Strong, high contrast
H2 / H3	Clear section hierarchy
Body	Readable and moderately compact
Caption / meta	Muted
Code	Monospace

3.5 Buttons

Type	Treatment
Primary	Blue or blue-to-purple gradient
Secondary	Dark outlined / glass button
Danger	Red outlined or filled
Success	Green visual treatment where appropriate

- Text must clearly describe the action.
- Do not use vague labels such as “Click Here”.
- Destructive actions require confirmation.

3.6 Common Statuses



Figure 5 – Shared status vocabulary

3.7 Standard UI States

Every major screen must support Loading, Empty, Error, Success, Not Found, Unauthorized, and Forbidden where applicable. AI-specific states are shown below.



Figure 6 – Standard and AI-specific UI states

3.8 Security UX

Frontend must never:

- Store provider secrets.
- Store Brevo credentials.
- Expose GitHub OAuth client secrets.
- Display internal AI provider credentials.

- Display raw backend exceptions.

The frontend must handle safe backend errors: errorCode, user-safe message, and requestId when returned.

4. Information Architecture

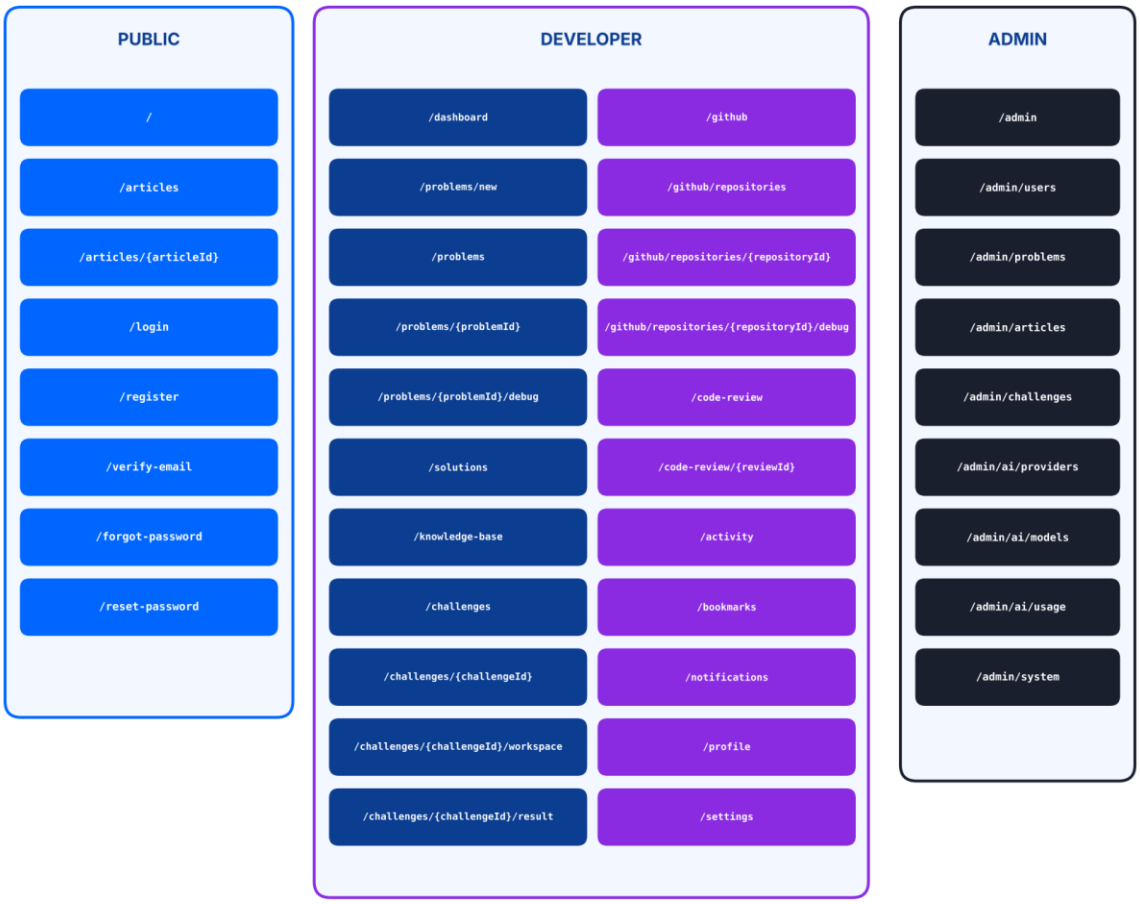


Figure 7 – Recommended route structure

> **Note**
> These route names are recommended frontend structure. Existing backend contracts remain authoritative for API paths.

5. Page Implementation Order

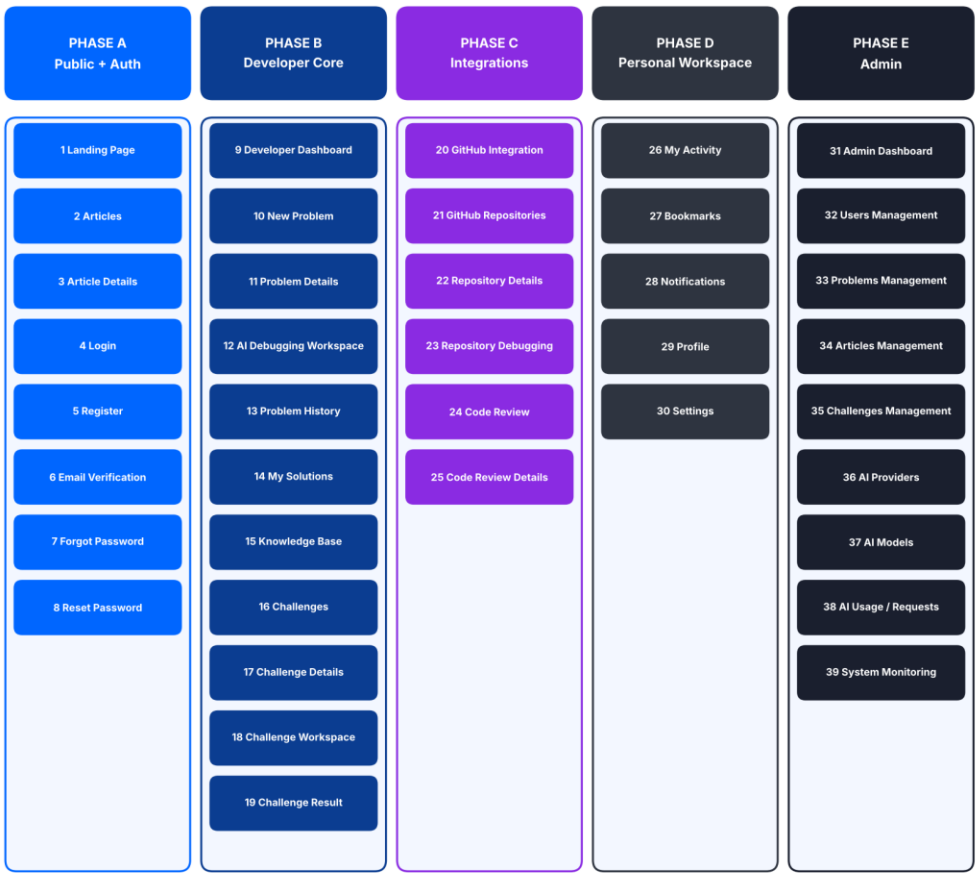


Figure 8 – 39 pages across five implementation phases

> **Reuse the approved language**
> The approved screenshots already establish the visual language for several pages. Remaining pages should reuse those components instead of inventing a new visual style.

6. Page Specifications

6.1 Landing Page

Access	Public
Purpose	Introduce DevPilot AI professionally and communicate the product value.
API	`GET /api/articles` `GET /api/challenges`

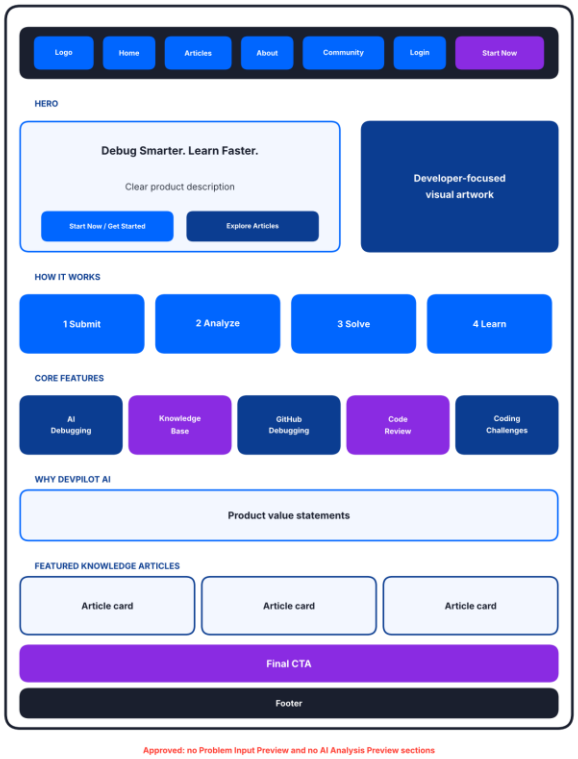


Figure 9 – Landing page structure

Approved visual decision

- Do NOT include: Problem Input Preview
- Do NOT include: AI Analysis Preview

Main structure

1. Header
2. Hero
3. How It Works
4. Core Features
5. Why DevPilot AI
6. Featured Knowledge Articles
7. Final CTA
8. Footer

Hero

- “Debug Smarter. Learn Faster.” style headline
- Clear product description
- CTA: Start Now / Get Started
- CTA: Explore Articles
- Developer-focused visual artwork

How It Works

1. Submit
2. Analyze
3. Solve
4. Learn

Core features

- AI Debugging
- Knowledge Base
- GitHub Debugging
- Code Review
- Coding Challenges

States

- Article preview loading
- Empty featured articles
- API failure with fallback content block

API note

- Mostly public content; no authenticated calls required for basic landing content.

> **Rule**
> Do not turn the page into a generic blog landing page.

6.2 Articles

Access	Public
Purpose	Public knowledge discovery.
API	`GET /api/articles`

Main structure

- Header
- Knowledge Articles hero
- Search
- Technology/category filters
- Sort
- Article cards

- Pagination

Article card

- Problem title
- Problem code
- Short description
- Technologies/tags
- Number of developers who faced it / relevant statistic
- Difficulty
- View article

Search support

- Keyword
- Error message
- Technology
- Problem code where configured

Recommended UI behavior

- Debounced search
- Preserve filter values
- Pagination state in URL where possible

> **Access**
> Public visitor can open any public article.

6.3 Article Details

Access	Public
Purpose	Present the full educational knowledge article generated from a solved developer problem.

API	`GET /api/articles/{articleId}`
	`GET /api/articles/{articleId}/versions (Administrator)`
	`GET /api/articles/{articleId}/versions/{versionId} (Administrator)`

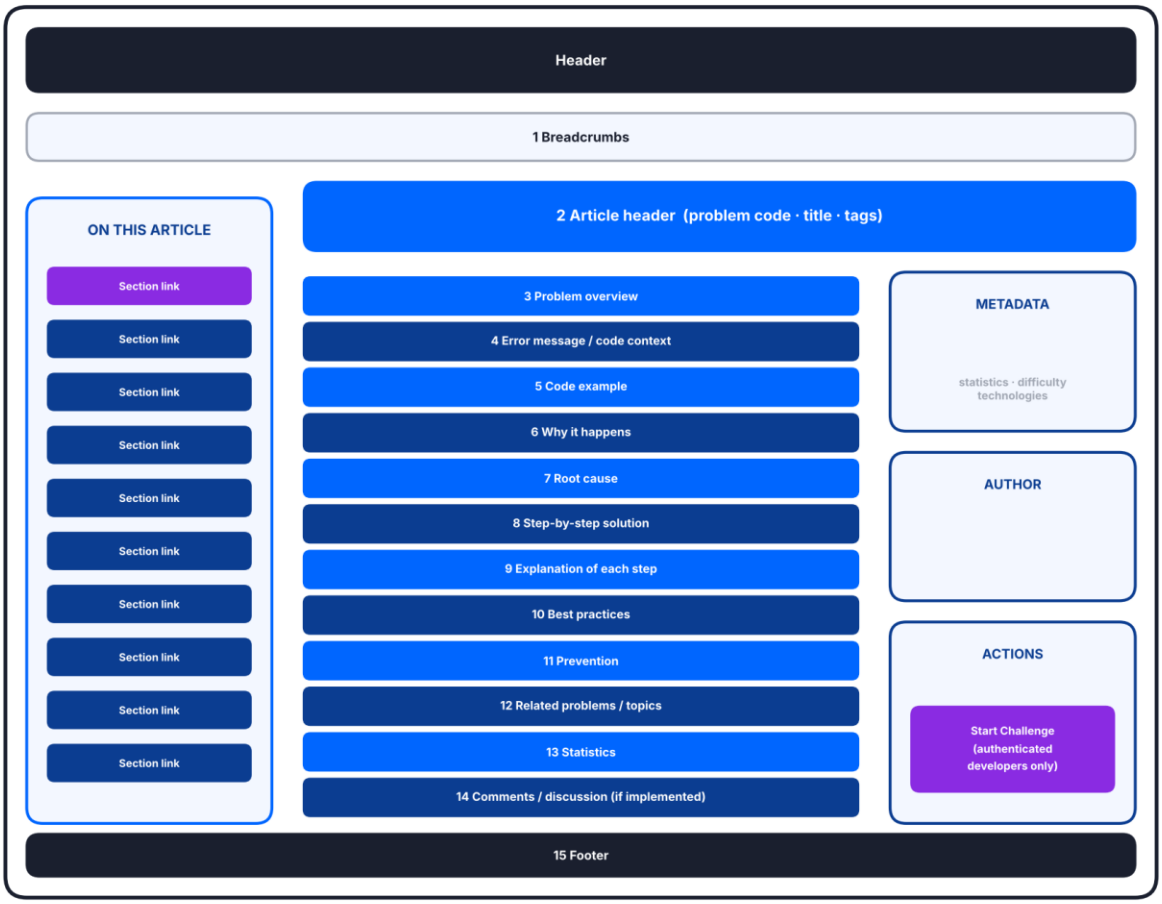


Figure 10 – Approved multi-part Article Details layout

Approved visual direction

- Use the approved multi-part Article Details design
- Left “On This Article” navigation
- Main article content
- Right metadata/author/actions panels
- Desktop only in the design phase

Content order

1. Breadcrumbs
2. Article header
3. Problem overview
4. Error message / code context
5. Code example
6. Why it happens
7. Root cause
8. Step-by-step solution
9. Explanation of each step
10. Best practices
11. Prevention

12. Related problems/topics
13. Statistics
14. Comments/discussion if implemented
15. Footer

Public visitor

- Can read article.
- Can view challenge information when available.
- Cannot start challenge.

Authenticated developer

- Can see Start Challenge on relevant article.
- Can navigate into Challenge Details / start flow.

Article response should support

- Problem code
- Problem
- Error/code context
- Root cause
- Step-by-step solution
- Explanation of each step
- Best practices
- Prevention
- Statistics
- Related problems
- Practice challenge when available

> **Approved business rule**
> [Start Challenge] is visible ONLY to authenticated developers. Public visitors must not be able to start the challenge.

6.4 Login

Access	Public
Purpose	Authenticate a verified developer with email and password only.
API	`POST /api/auth/login`

Approved rules

- Email + Password only
- No “Continue with...”
- No Google/GitHub/social login section
- No social authentication UI

Structure

- Header

- Split visual area
- Left DevPilot value proposition
- Right login card
- Email
- Password
- Remember me
- Forgot password
- Login
- Create Account
- Footer

Login UX

- Client-side required validation
- Disable button while submitting
- Show safe invalid-credentials message
- Handle unverified email response
- Provide path to resend verification

> **Rules**
> Do not add social authentication. Remember-me behavior must follow the backend/session strategy; do not invent token storage behavior outside the approved authentication design.

6.5 Register

Access	Public
Purpose	Create a developer account and start email verification.
API	`POST /api/auth/register`

Fields

- Name
- Email
- Password

Registration process

1. Validate input
2. Create user
3. User is unverified
4. Verification record is created
5. Email sent through Brevo
6. Frontend moves to Email Verification flow

Frontend validation

- Name required
- Valid email

- Strong password
- Show password visibility
- Show safe validation messages

> **Important**
> Email verification is required before normal authenticated access. No social signup is part of the approved flow; do not implement Google/GitHub signup unless explicitly approved later.

6.6 Email Verification

Access	Public during verification flow
Purpose	Verify the developer's email with the approved dark DevPilot visual and code entry card.
API	`POST /api/auth/verify-email` `POST /api/auth/resend-verification`

UI

- Email being verified
- Six-digit OTP input when OTP mode is used
- Verification button
- Expiration countdown
- Resend Verification
- Change email
- Helpful instructions

Rules

- Show resend cooldown
- Prevent excessive submit attempts
- Show expired-code state
- Show success confirmation
- Do not expose raw verification token in UI

Success

- Redirect to authenticated app / dashboard after successful verification.

> **Backend method**
> Backend supports OTP or verification link. Frontend should follow the currently selected backend method.

6.7 Forgot Password

Access	Public
Purpose	Start the password reset flow.
API	`POST /api/auth/forgot-password`

UI

- Email field
- Send Reset Link
- Back to Login
- Security guidance

States

- Invalid email format
- Submitting
- Success
- Rate limited
- Generic failure

> **Security rule**
> The UI should not reveal whether an email exists. Recommended success message: “If the account exists, a password reset email has been sent.”

6.8 Reset Password

Access	Public with valid reset token
Purpose	Set a new password using a valid reset token.
API	`POST /api/auth/reset-password`

Fields

- New Password
- Confirm New Password

Display

- Password strength
- Password requirements
- Reset Password
- Back to Login

States

- Valid token
- Expired/invalid token
- Password mismatch
- Weak password
- Success

After success

- Show reset success
- Return user to Login

6.9 Developer Dashboard

Access	Verified Developer
Purpose	Primary command center for the developer, focused on DevPilot's true value: problems, AI debugging, reusable solutions, knowledge, challenges, GitHub and code review.

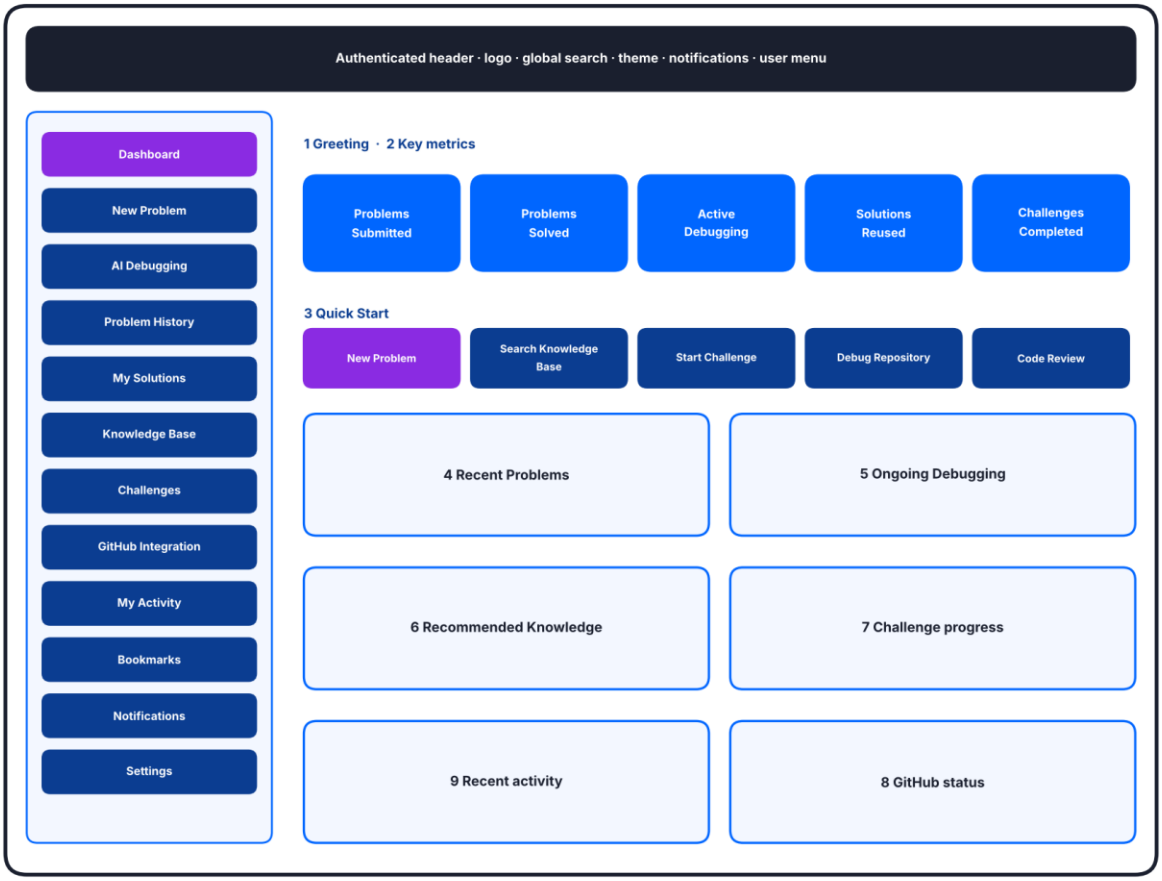


Figure 11 – Developer dashboard layout

Recommended sections

1. Greeting
2. Key metrics
3. Quick Start
4. Recent Problems
5. Ongoing Debugging
6. Recommended Knowledge
7. Challenge progress
8. GitHub status
9. Recent activity
10. Optional activity/progress widgets

Key metrics

- Problems Submitted

- Problems Solved
- Active Debugging
- Solutions Reused
- Challenges Completed

Quick actions

- New Problem
- Search Knowledge Base
- Start Challenge
- Debug Repository
- Code Review

> **Rules**
> Do NOT focus the dashboard on article-reading metrics. Data examples should be real backend data, not hardcoded demo stats.

6.10 New Problem / Submit Problem

Access	Verified Developer
Purpose	Create a technical problem for AI debugging.

API	`POST /api/problems`
	`POST /api/problems/{problemId}/requests`
	`POST /api/problems/{problemId}/debug`

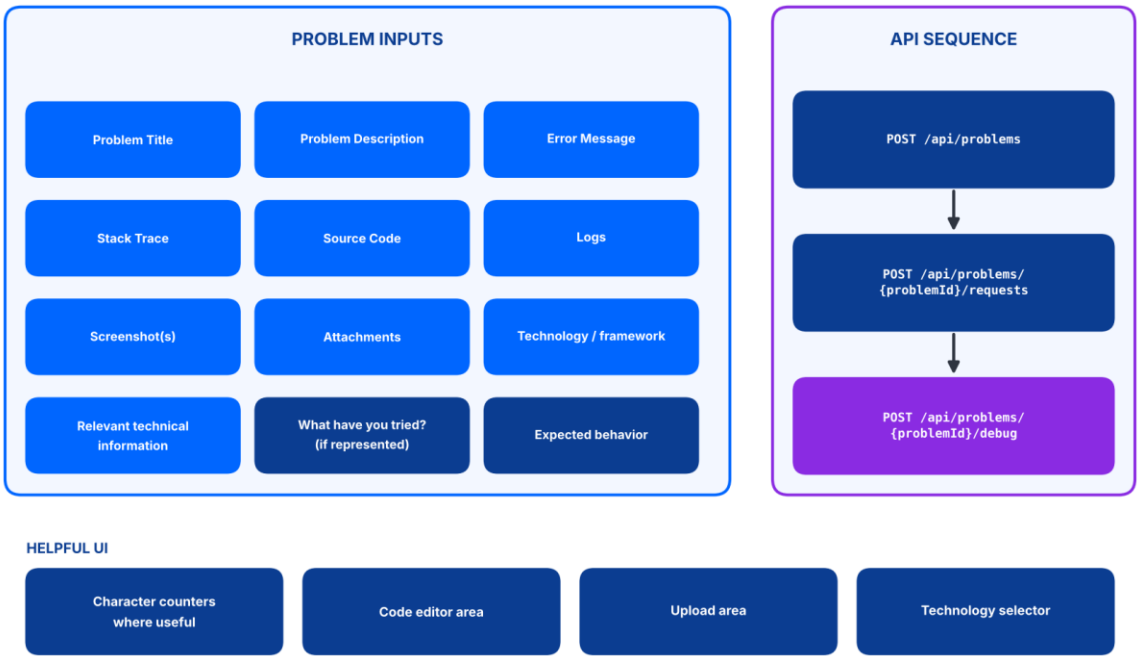


Figure 12 – Problem inputs and API sequence

Inputs

- Problem Title
- Problem Description
- Error Message
- Stack Trace
- Source Code
- Logs
- Screenshot(s)
- Attachments
- Technology / framework
- Relevant technical information

Helpful UI

- Character counters where useful
- Code editor area
- Upload area
- Technology selector
- “What have you tried?” area if represented
- Expected behavior

> **Core design principle**
> The form should encourage useful context because better context produces better debugging. Do not make fields mandatory that the backend/business considers optional, except genuinely required fields.

6.11 Problem Details

Access	Problem owner
Purpose	Show the complete record of one problem without turning this screen into the actual AI Debugging Workspace.
API	`GET /api/problems/{problemCode}` `GET /api/problems/{problemId}/history`

Important distinction

- Problem Details = historical/problem record page.
- AI Debugging Workspace = active AI interaction page.

Must contain

- Problem code
- Title
- Status
- Technology/tags
- Original request
- Error/stack trace
- Code/logs/attachments

- Debugging attempts timeline
- Developer feedback history
- Current/selected solution summary
- Actions to continue debugging

History should expose

- requests
- attempts
- feedback
- successful solutions

> **Rules**
> Do NOT overload this page with the complete AI conversation. Never remove previous attempts.

6.12 AI Debugging Workspace

Access	Problem owner
Purpose	Core product experience: structured, iterative AI debugging with explicit developer feedback.
API	`POST /api/problems/{problemId}/debug` `POST /api/problems/{problemId}/attempts/{attemptId}/feedback`

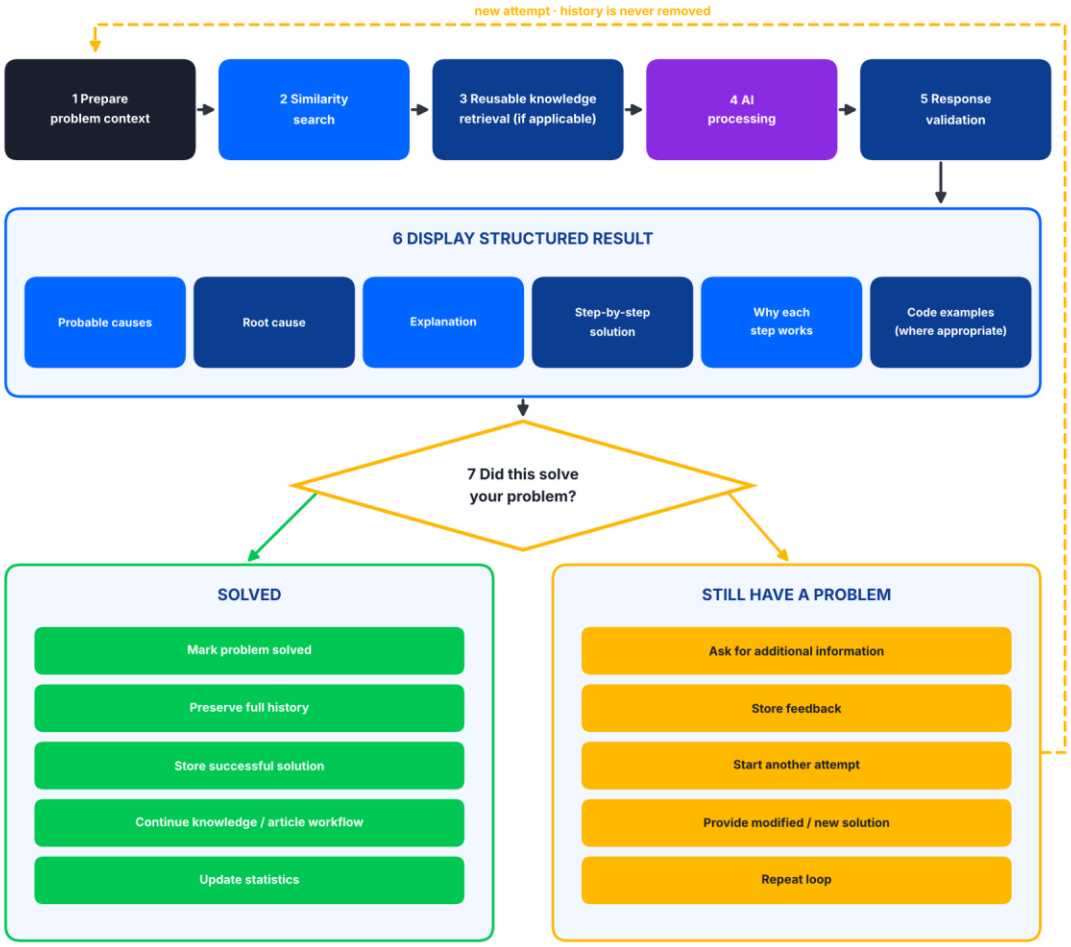


Figure 13 – AI debugging workspace flow and feedback loop

Required flow

1. Prepare problem context
2. Similarity search
3. Reusable knowledge retrieval when applicable
4. AI processing
5. Response validation
6. Display probable causes, root cause, explanation, step-by-step solution, why each step works, code examples where appropriate
7. Ask: “Did this solve your problem?”
8. Buttons: Solved / Still Have a Problem

If Solved

- Mark problem solved
- Preserve full history
- Store successful solution
- Continue knowledge/article workflow
- Update statistics

If Still Have a Problem

- Ask for additional information
- Store feedback
- Start another attempt
- Provide modified/new solution
- Repeat loop

AI progress UI

- Preparing
- Searching similar problems
- Reusing knowledge
- Analyzing
- Generating solution
- Validating response

> **Rules**
> The UI must make the iterative nature of debugging obvious. Do not show an answer as “certain” when backend/AI only identifies a probable cause.

6.13 Problem History

Access	Verified Developer
Purpose	List all developer-owned problems and allow continuation.

Features

- Search
- Filters
- Status tabs
- Technology filter
- Date filter
- Problem code
- Attempts count
- Last updated
- Open action

Each row/card

- Problem title
- DEV code
- Technology
- Status
- Attempts
- Last updated
- Open

Possible summary metrics

- Total Problems
- Solved
- In Progress
- Needs Feedback

> **Rule**
> Do not transform this page into a generic activity page.

6.14 My Solutions

Access	Verified Developer
Purpose	Show successful solutions produced from solved debugging sessions. The developer can inspect solutions they have already proven successful.

Data

- Solution code/ID where available
- Original problem
- Technology
- Summary
- Reuse count
- Related knowledge article
- Last used
- View solution
- Reuse/adapt action where business flow supports it

Relevant database

- ProblemSolutions
- KnowledgeArticles
- SimilarityMatches

> **Important**
> This is NOT a manual article-writing page. Each solution should represent a proven or successfully completed resolution.

6.15 Knowledge Base – Developer View

Access	Public browsing + authenticated enhancements
Purpose	Search and reuse solved developer knowledge.
API	`GET /api/articles` `POST /api/search/similar-problems (authenticated)`

Capabilities

- Search
- Technology filter
- Tag filter
- Problem code
- Error message
- Semantic similarity
- Related problems
- Recommended knowledge

Results

- Problem title
- Problem code
- Short description
- Technologies/tags
- Developer count/statistics
- Relevance

Optional personalized enhancements

- Related to your problems
- Recommended knowledge
- Recently viewed
- Saved/bookmarked

> **Rule**
> Do not expose incomplete internal article versioning data to normal developers.

6.16 Challenges

Access	Browse = Public · Start = Verified Developer
Purpose	Practice real-world coding challenges generated from solved knowledge.

API	`GET /api/challenges`
	`GET /api/challenges/{challengeId}`
	`POST /api/challenges/{challengeId}/start`

Public user

- Can browse challenge listings/details.
- Cannot start a challenge.

Developer

- Can start and track challenges.

Challenge listing

- Title
- Technologies

- Difficulty
- Short objective
- Estimated time when available
- Participation/statistics when available
- Start/View details actions according to authorization

> **Rule**
> Do NOT expose the complete challenge solution in public challenge details.

6.17 Challenge Details

Access	Public view of challenge metadata · Start action = Verified Developer
Purpose	Explain a challenge without revealing the final solution.
API	`GET /api/challenges/{challengeId}` `POST /api/challenges/{challengeId}/start`

Sections

- Challenge title
- Based-on problem when available
- Objective
- Requirements
- Technologies
- Difficulty
- Expected behavior
- Evaluation criteria
- Rules
- Start Challenge

> **Important**
> Start Challenge appears only for authenticated developers. The challenge should be understandable without revealing the final solution.

6.18 Challenge Workspace

Access	Challenge attempt owner
Purpose	Track the developer while they implement the challenge.

API	`GET /api/challenges/attempts/{attemptId}`
	`POST /api/challenges/attempts/{attemptId}/evaluate`
	`POST /api/github/challenges/{challengeId}/repository`

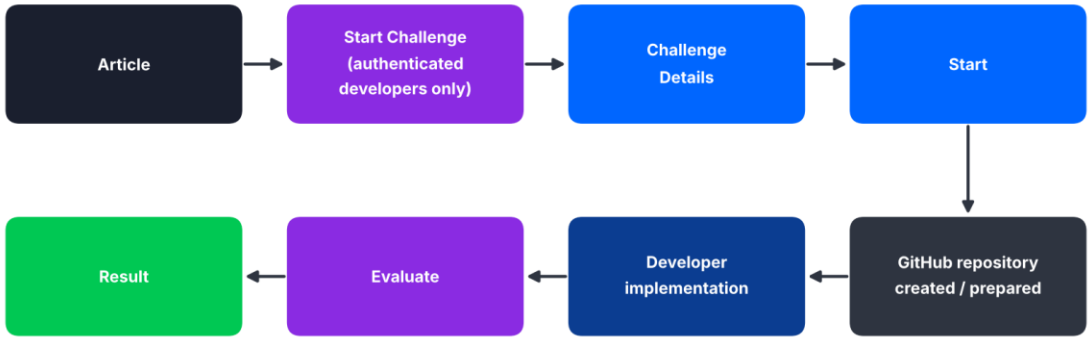


Figure 14 – Challenge lifecycle from article to result

Possible sections

- Challenge requirements
- Repository
- Progress
- Test/build status
- Submission/evaluation state
- Repository link
- Evaluate action

Backend process

- ChallengeAttempt is created
- GitHub repository can be created/prepared
- Repository is associated with the attempt

> **Note**
> The online coding environment itself is not required to be a full custom IDE.

6.19 Challenge Result

Access	Attempt owner
Purpose	Show final challenge evaluation.

Evaluation dimensions

- Functional correctness
- Requirement completion
- Code quality
- Security
- Performance

- Architecture
- Understanding
- Best practices

UI

- Overall score
- Category scores
- Build/test results
- AI evaluation summary
- Strengths
- Weaknesses
- Recommendations
- Repository action

> **Rule**
> Never fake evaluation scores in the production UI.

6.20 GitHub Integration

Access	Verified Developer
Purpose	Connect and manage GitHub.

API	`GET /api/github/connect`
	`GET /api/github/callback`
	`GET /api/github/repositories`
	`POST /api/github/webhook`

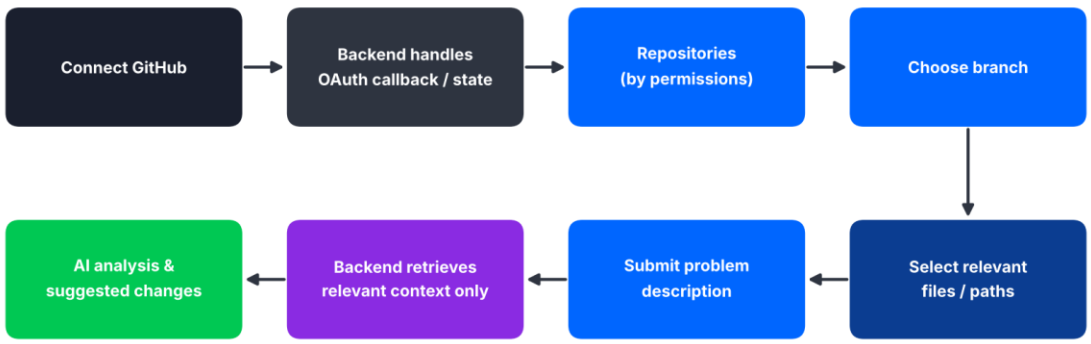


Figure 15 – Repository debugging flow

Business capabilities

- OAuth connection
- Repository listing

- Branch selection
- File/folder browsing
- Repository debugging
- Code review
- Challenge repository workflow
- Webhooks

Integration page sections

- Connection status
- GitHub account information
- Sync status
- Repository statistics
- Recent repositories
- Quick actions
- Recent activity

Actions

- Connect / Disconnect
- Sync
- Open repositories
- Debug repository
- Code review

Security

- Never display the raw GitHub access token.
- OAuth state must be handled by backend.
- Frontend only receives safe account/status information.

6.21 GitHub Repositories

Access	Verified Developer
Purpose	Browse repositories available through GitHub permissions.
API	`GET /api/github/repositories`

Features

- Search repositories
- Filter
- Branch display/selection
- Language/technology
- Last updated
- Debug
- Review

> **Rule**
> Do not assume every repository is public. Private repository access depends on GitHub permissions and backend authorization.

6.22 Repository Details

Access	Verified Developer with repository access
Purpose	Inspect a repository before running debugging/review.
API	`GET /api/github/repositories/{repositoryId}/files (query: branch, path)`

Sections

- Repository name
- Owner
- Visibility
- Default branch
- Branch selector
- File tree
- Recent commits
- Languages
- Repository metadata
- Debug Repository
- Code Review

> **Rule**
> Only retrieve relevant content when needed.

6.23 Repository Debugging

Access	Verified Developer
Purpose	Debug a real repository using selected repository context.
API	`POST /api/github/repositories/{repositoryId}/debug`

Input

- Repository
- Branch
- Problem description
- Selected paths/files

UI

- Problem description
- Branch
- Selected files

- Context size/summary where useful
- AI analysis
- Root cause
- Affected files
- Suggested changes
- Code explanation

Important context optimization
Do not send the entire repository to the LLM. The business/AI architecture explicitly prioritizes only relevant files/context.

6.24 Code Review

Access	Verified Developer
Purpose	Request an AI code review for a repository, branch, PR or files.
API	`POST /api/code-review`



Figure 16 – Code review request flow, severities and finding categories

Input examples

- repository
- branch
- pull request
- selected files

Review may identify

- Bugs
- Security issues

- Performance issues
- Code smells
- SOLID concerns
- Maintainability
- Architecture concerns
- Best-practice issues

UI

- Repository selector
- Branch/PR selector
- Optional file selection
- Review settings
- Start Review
- Review status

6.25 Code Review Details

Access	Review owner / Administrator
Purpose	Show the structured findings of a review.
API	`GET /api/code-review/{reviewId}`

Finding

- File path
- Line
- Severity
- Category
- Problem
- Suggestion

UI

- Summary counts by severity
- File/line context
- Finding details
- Suggested fix
- Before/after code where backend provides it

Severity should be visually obvious

- Critical
- High
- Medium
- Low

> **Rule**

> Do not make the frontend claim severity categories that the backend does not actually return. Map backend values consistently.

6.26 My Activity

Access	Verified Developer
Purpose	Personal activity center.

Activity types (map to actual platform events)

- Problem submitted
- Debugging attempt
- Problem solved
- Solution reused
- Challenge started/completed
- GitHub connected/synced
- Repository debugging
- Code review
- Knowledge interaction

Recommended structure

- Summary metrics
- Activity chart or timeline
- Recent activity
- Recent contributions
- Technology usage
- Streak/progress only where meaningful

> **Important**
> Activity is secondary to the core business. Do not turn My Activity into a social-media feed.

6.27 Bookmarks

Access	Verified Developer
Purpose	Save useful developer knowledge for quick access.

Can contain saved

- Problems
- Solutions
- Knowledge/articles
- Relevant resources represented by backend

Features

- Tabs/categories

- Search
- Technology filter
- Tags
- Sort
- Remove bookmark
- Open item

> **Business alignment**
> Bookmarks are for knowledge reuse and developer workflow, not social posting.

6.28 Notifications

Access	Verified Developer
Purpose	Present important in-app updates.

Possible notification types

- Challenge created
- Challenge evaluation completed
- GitHub event received
- Article updated
- New related knowledge
- System notification

Channels (depending on integration state)

- In-app
- Telegram
- Discord

UI

- Unread count
- All / category tabs
- Mark as read
- Mark all as read
- Notification list
- Date grouping
- Deep link to related entity

> **Rule**
> Do not invent unsupported notification types.

6.29 Profile

Access	Verified Developer
Purpose	Manage basic developer profile.
API	`GET /api/users/me` `PATCH /api/users/me`

Recommended content

- Name
- Email (with verification status)
- Profile image if supported
- Developer summary/bio if supported
- Connected GitHub status
- Personal statistics

> **Rule**
> The profile must not expose internal security fields.

6.30 Settings

Access	Verified Developer
Purpose	Manage account and product preferences.

Approved setting areas

- Account
- Security
- Notifications
- GitHub Integration
- AI Preferences

Area	Recommended content
Account	Basic account information
Security	Password change · sessions/security controls where backend supports them
Notifications	Notification preferences
GitHub	Connection status · manage integration
AI Preferences	Only preferences that have real backend behavior. Do not invent provider keys or unsafe controls.

Potential account actions

- Download/inspect own data if supported
- Delete account where backend supports it

Security note

Do not create fake billing/subscription controls unless subscription/plan requirements are officially added to the backend/business scope.

7. Admin Page Specifications



Figure 17 – Administrator pages and responsibilities

7.1 Admin Dashboard

Access	Administrator only

- Purpose: system overview.
- Potential metrics: Users, Problems, Solved problems, Knowledge articles, Challenges, AI requests, Provider health, Errors, GitHub events

> **Rule**
> Use real backend data.

7.2 Users Management

Access	Administrator

- Search users
- View user
- Verify status
- Role
- Created date
- Account status
- Safe account actions

> **Rule**
> Do not expose password hashes or security secrets.

7.3 Problems Management

Access	Administrator

- Search problems
- Filter status
- Technology
- Problem code
- View problem
- View debugging history
- Knowledge status
- Statistics

> **Note**
> Administrator can inspect history and knowledge generation status.

7.4 Articles Management

Access	Administrator

API	`GET /api/articles/{articleId}/versions`
	`GET /api/articles/{articleId}/versions/{versionId}`
	`POST /api/problems/{problemId}/article/generate`
	`POST /api/articles/{articleId}/regenerate`

- Search articles
- Publish/unpublish where supported
- View current version
- View previous versions
- Regenerate / update article where backend workflow allows

> **Rule**
> The frontend should clearly distinguish: Current version vs Historical version.

7.5 Challenges Management

Access	Administrator

- Review generated challenges
- View challenge status
- View participation
- Inspect challenge configuration
- Monitor evaluation workflow

> **Rule**
> Do not expose complete answers unnecessarily.

7.6 AI Providers

Access	Administrator

- Purpose: monitor/configure configured AI providers.
- Provider name
- Status
- Health
- Limits/usage metadata where returned
- Cooldown/failure state
- Active/inactive state where supported

> **Rules**
> Never show actual API keys. The frontend should not assume one provider; the AI Gateway handles provider selection/failover.

7.7 AI Models

Access	Administrator

- Provider
- Model
- Enabled state
- Task suitability
- Health/availability
- Usage metadata

> **Note**
> The system should support provider independence.

7.8 AI Usage / Requests

Access	Administrator

- Purpose: monitor AI request utilization.
- Metrics: Request count, Success/failure, Token usage, Provider/model, Task type, Latency, Retry/failover, Time window

> **Rule**
> Do not show sensitive prompt/code payloads unless specifically authorized.

7.9 System Monitoring

Access	Administrator

- Health areas: API, Database, AI Gateway, AI providers, GitHub, Email delivery, Telegram/Discord integrations

- Health states: Healthy, Degraded, Down, Unknown

> **Rule**
> Use clear health states.

8. Core Shared Components

The frontend developer must create reusable components.

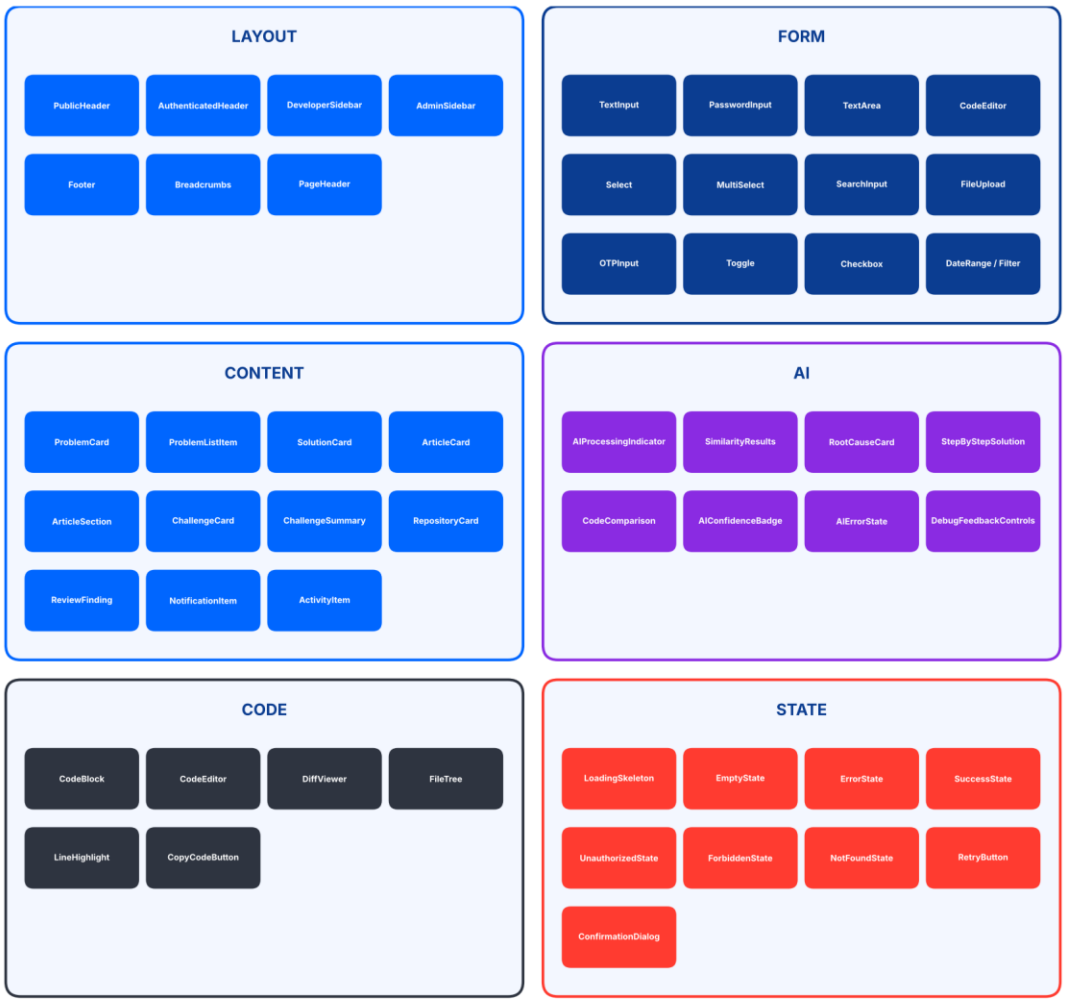


Figure 18 – Reusable component library

Group	Components
8.1 Layout	PublicHeader, AuthenticatedHeader, DeveloperSidebar, AdminSidebar, Footer, Breadcrumbs, PageHeader
8.2 Form	TextInput, PasswordInput, TextArea, CodeEditor, Select, MultiSelect, SearchInput, FileUpload, OTPInput, Toggle, Checkbox, DateRange/Filter controls
8.3 Content	ProblemCard, ProblemListItem, SolutionCard, ArticleCard, ArticleSection, ChallengeCard, ChallengeSummary, RepositoryCard, ReviewFinding, NotificationItem, ActivityItem
8.4 AI	AIProcessingIndicator, SimilarityResults, RootCauseCard, StepByStepSolution, CodeComparison, AIConfidenceBadge, AIErrorState, DebugFeedbackControls
8.5 Code	CodeBlock, CodeEditor, DiffViewer, FileTree, LineHighlight, CopyCodeButton
8.6 State	LoadingSkeleton, EmptyState, ErrorState, SuccessState, UnauthorizedState, ForbiddenState, NotFoundState, RetryButton, ConfirmationDialog

9. API Integration Map

Auth

Method	Endpoint	Note
POST	/api/auth/register	
POST	/api/auth/verify-email	
POST	/api/auth/resend-verification	
POST	/api/auth/login	
POST	/api/auth/refresh	
POST	/api/auth/logout	
POST	/api/auth/forgot-password	
POST	/api/auth/reset-password	

User

Method	Endpoint	Note
GET	/api/users/me	
PATCH	/api/users/me	
GET	/api/users/me/problems	
GET	/api/users/me/challenges	
GET	/api/users/me/integrations	

Problems

Method	Endpoint	Note
POST	/api/problems	
GET	/api/problems/{problemId or problemCode}	
POST	/api/problems/{problemId}/requests	
POST	/api/problems/{problemId}/debug	
POST	/api/problems/{problemId}/attempts/{attemptId}/feedback	
GET	/api/problems/{problemId}/history	
GET	/api/problems/search	

Search

Method	Endpoint	Note
POST	/api/search/similar-problems	

Articles

Method	Endpoint	Note
GET	/api/articles	
GET	/api/articles/{articleId}	
GET	/api/articles/{articleId}/versions	Admin
GET	/api/articles/{articleId}/versions/{versionId}	Admin

Challenges

Method	Endpoint	Note
GET	/api/challenges	
GET	/api/challenges/{challengeId}	
POST	/api/challenges/{challengeId}/start	
GET	/api/challenges/attempts/{attemptId}	
POST	/api/challenges/attempts/{attemptId}/evaluate	

GitHub

Method	Endpoint	Note
GET	/api/github/connect	
GET	/api/github/callback	
GET	/api/github/repositories	
GET	/api/github/repositories/{repositoryId}/files	
POST	/api/github/repositories/{repositoryId}/debug	
POST	/api/github/challenges/{challengeId}/repository	
POST	/api/github/webhook	

Code Review

Method	Endpoint	Note
POST	/api/code-review	
GET	/api/code-review/{reviewId}	

Telegram

Method	Endpoint	Note
POST	`/api/telegram/webhook`	

AI Gateway / Admin

Method	Endpoint	Note
POST	`/api/ai/generate`	Admin provider/model/usage endpoints: use the exact backend contract when implemented.

10. Authentication Flow



Figure 19 – Register, login, forgot-password and logout flows

Frontend must centralize auth handling:

- Auth provider/store
- Route guards
- Token refresh handling
- Logout handling
- Session-expired handling

11. Route Guards

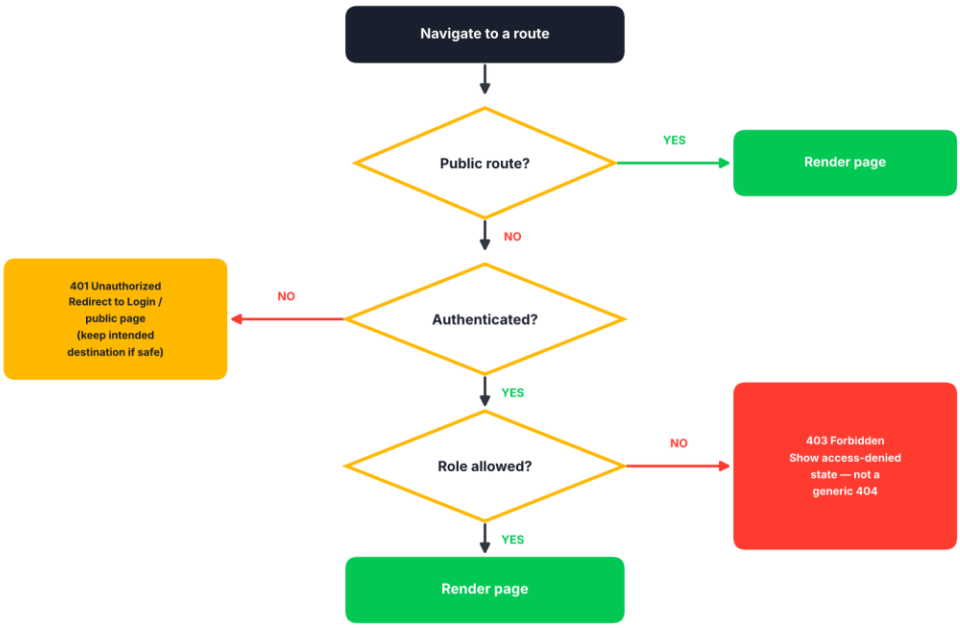


Figure 20 – Route guard decision flow

Public	Authenticated Developer	Admin
Landing, Articles, Article Details, Login, Register, Verify Email, Forgot Password, Reset Password	Dashboard, New Problem, Problem Details, AI Debugging, Problem History, My Solutions, Developer Knowledge Base enhancements, Challenge Start/Attempt, GitHub, Code Review, Activity, Bookmarks, Notifications, Profile, Settings	All /admin routes

Case	Behavior
Unauthorized	Redirect to Login or public page. Preserve intended destination when safe.
Forbidden	Show access-denied state, not generic 404.

12. AI UX Requirements

> **Principle**
> The AI is not a generic chat box. The UI should expose structured AI output.



Figure 21 – AI interaction states

Required structured concepts

- Problem understanding
- Possible causes
- Probable root cause
- Why it happens
- Step-by-step solution
- Why each step works
- Code example/fix when appropriate
- Best practices
- Prevention
- Similar problems / reused knowledge
- Feedback loop

Token optimization should be invisible as technical implementation but visible as useful UX when appropriate, for example: “Found a similar solved problem” or “Adapting a proven solution”. Never expose raw provider selection details to normal users.

13. Knowledge Reuse UX

Before every fresh AI call, the backend may search similar problems. Frontend may show:

- Similar problem found

- Match percentage/relevance if the backend returns it
- Previous successful solution
- “Adapted from existing knowledge”

> **Rules**
> Do not show similarity scores if the backend does not return them. When a new solution is successful it becomes reusable knowledge, may update/create an article, and may be considered for future similar problems.

The UX should communicate: “Your solved problem becomes future developer knowledge.”

14. Article Evolution UX

Articles can improve over time. The system may store multiple successful solution variants, synthesize better knowledge, create a new article version, and keep previous versions.

Audience	Sees
Normal developers	The current public article.
Administrators	Can inspect version history.

> **Rule**
> Do not expose draft/internal article versions to the public.

15. GitHub UX Rules

Area	Rules
OAuth	Start from Connect GitHub · Backend handles OAuth callback/state · Frontend displays resulting connection state
Repository debugging	User chooses repository → chooses branch → selects relevant files/paths → submits problem → backend retrieves relevant context
Code Review	Choose repo → choose branch or PR → start review → review findings → open exact file/line
Challenge	Article → Start Challenge → Challenge Details → Start → GitHub repository → Developer implementation → Evaluation → Result

> **Important**
> The frontend should never send the whole repository content itself unless the backend explicitly requires selected file content.

16. File / Attachment UX

Supported problem attachments may include screenshots, logs, source/code files and relevant technical files. The UI must include:

- Drag/drop
- Upload progress
- File name
- File size
- File type
- Remove file
- Upload error

Security

- Respect backend file validation
- Do not execute files
- Do not show internal storage paths

17. Error Handling

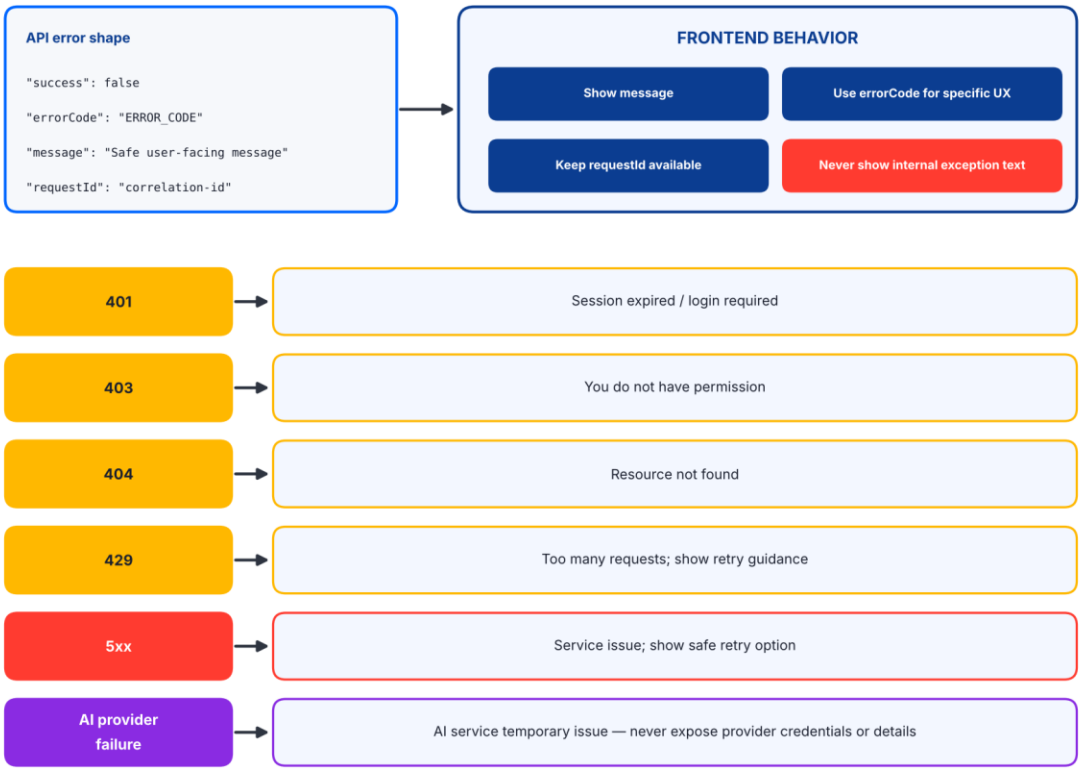


Figure 22 – Safe error handling

Expected API error shape:

```
{
  "success": false,
  "errorCode": "ERROR_CODE",
  "message": "Safe user-facing message",
  "requestId": "correlation-id"
}
```

18. Loading and Performance

- Skeletons for dashboard/card lists
- Spinners only for short actions
- Progressive loading for long pages
- Debounced searches
- Pagination for large datasets
- Lazy load heavy code areas
- Do not fetch repository files until needed
- Do not render large code blocks unnecessarily

AI debugging: a long-running action should show a progress state, and the UI must remain usable while waiting.

19. Accessibility

Minimum requirements:

- Keyboard navigation
- Visible focus state
- Form labels
- Error associations
- Buttons must have readable labels
- Icon-only buttons require accessible labels
- Code blocks must remain readable
- Color must not be the only way to communicate status
- Sufficient contrast for text

20. Data Display Rules

Use real backend values. Never hardcode:

- Problem counts
- Solved counts
- Challenge scores
- AI confidence
- GitHub repo counts
- Notification counts
- Usage statistics

Demo/mock data may be used only during design/prototyping and must be isolated from production data code.

Topic	Rule
Dates	Use consistent formatting.
Counts	Use compact formatting only when appropriate (e.g. 2.1K).
Statuses	Use shared status badges.

21. Search and Filter Rules

Search should preserve: Query, Technology, Category, Status, Date, Sort. Use URL query parameters for shareable/list pages where practical.

Page	Search scope
Public articles	Title, description, error, technology
Problems	Title, error, problem code
GitHub	Repository name
Challenges	Title, technology
Bookmarks	Saved item title/content

22. Dashboard Data Relationship

Dashboard widgets must map to actual business objects.



Figure 23 – Dashboard widgets and their business-object sources

23. Frontend Security Checklist

DO	DO NOT
Centralize authentication state.	Put API keys in frontend source.
Protect private routes.	Put Brevo credentials in frontend.
Validate forms client-side and trust backend validation as authoritative.	Put GitHub client secrets in frontend.
Sanitize/display backend content safely.	Put LLM provider keys in frontend.
Protect against accidental token leakage.	Trust frontend authorization alone.
Handle session expiration.	Display internal exceptions.
Use secure API communication.	Assume public/private state from UI alone.
Keep secrets out of frontend builds.	

24. UX Do’s and Don’ts

UX DO	UX DON’T
Keep the developer/problem workflow obvious.	Turn DevPilot AI into a generic chatbot.
Use structured AI output.	Turn the Dashboard into a reading-only dashboard.
Make code readable.	Add unsupported product features.
Make debugging progress visible.	Add social login to Login.
Make “Solved / Still Have a Problem” very clear.	Show Start Challenge to unauthenticated visitors.
Preserve history.	Expose private repository information publicly.
Surface useful similar knowledge.	Show internal AI provider details.
Make challenge entry contextual from solved articles/problems.	Show fake metrics in production.
Reuse shared components.	Replace the approved logo/theme.
Keep screens dense but organized.	Create unrelated page styles.
	Hide the debugging history.
	Treat one AI response as final before developer feedback.

26. Design System Implementation

Recommended tokens. The swatches below map each token role to the supplied Mahmoud Abou-Zaeid palette for this document; replace them with the approved DevPilot AI design values if they differ.

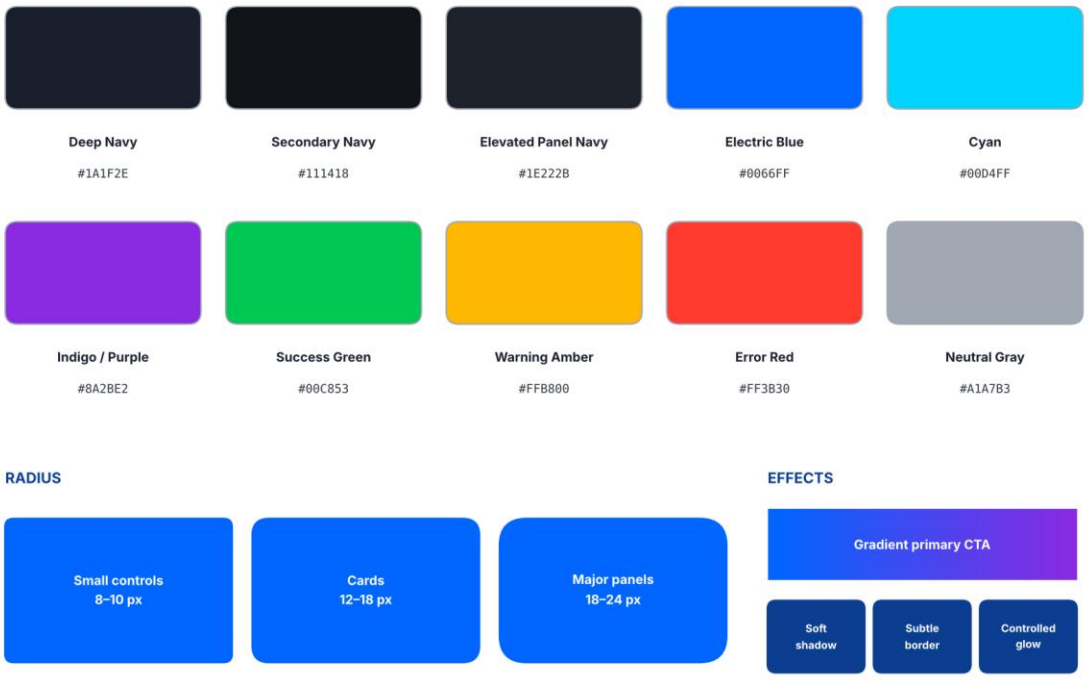


Figure 24 – Recommended design tokens

> **Avoid excessive glassmorphism**
> Use it as an accent, not everywhere.

27. Page-Level State Matrix

Page	States
Landing	Normal / Loading / API Error
Articles	Normal / Loading / Empty / Search Empty / Error
Article Details	Loading / Loaded / Not Found / Error. Challenge CTA: hidden for public; visible for authenticated developer when challenge exists
Login	Idle / Validation Error / Loading / Invalid Credentials / Unverified / Server Error / Session Created
Register	Idle / Validation Error / Loading / Email Exists / Success / Server Error
Verify Email	Waiting / Invalid Code / Expired / Resend Cooldown / Success / Error
Forgot Password	Idle / Loading / Success / Rate Limited / Error
Reset Password	Idle / Weak Password / Mismatch / Invalid Token / Success / Error
Dashboard	Loading / Loaded / Empty / Partial Data Error / Global Error
New Problem	Idle / Uploading / Validating / Creating / Created / Error
Problem Details	Loading / Loaded / Not Found / Error
AI Debugging	Preparing / Similarity Search / Analyzing / Validating / Result / Waiting for Feedback / Retrying / Solved / Failed
Problem History	Loading / Empty / Filtered Empty / Error
My Solutions	Loading / Empty / Search Empty / Error
Knowledge Base	Loading / Empty / No Results / Error
Challenges	Loading / Empty / Filtered Empty / Error
Challenge Details	Loading / Not Found / Available / Start Disabled / Start
Challenge Workspace	Loading / Ready / Syncing / Test Running / Evaluation Pending / Error
Challenge Result	Pending / Evaluated / Partial Results / Error
GitHub	Not Connected / Connecting / Connected / Syncing / Error
Repositories	Loading / Empty / Search Empty / Error
Repository Details	Loading / Ready / Permission Error / Not Found
Repository Debug	Preparing / Loading Context / AI Analysis / Result / Error
Code Review	Selecting / Running / Completed / Failed
Code Review Details	Loading / Loaded / Not Found / Error
Activity	Loading / Empty / Filtered Empty / Error
Bookmarks	Loading / Empty / Search Empty / Error
Notifications	Loading / Empty / Unread / Read / Error
Profile	Loading / Editing / Saving / Success / Error
Settings	Loading / Saving / Success / Error

Page	States
Admin	Loading / Unauthorized / Empty / Error. Role guard must always run before rendering protected admin data.

28. Implementation Notes for the Frontend Developer

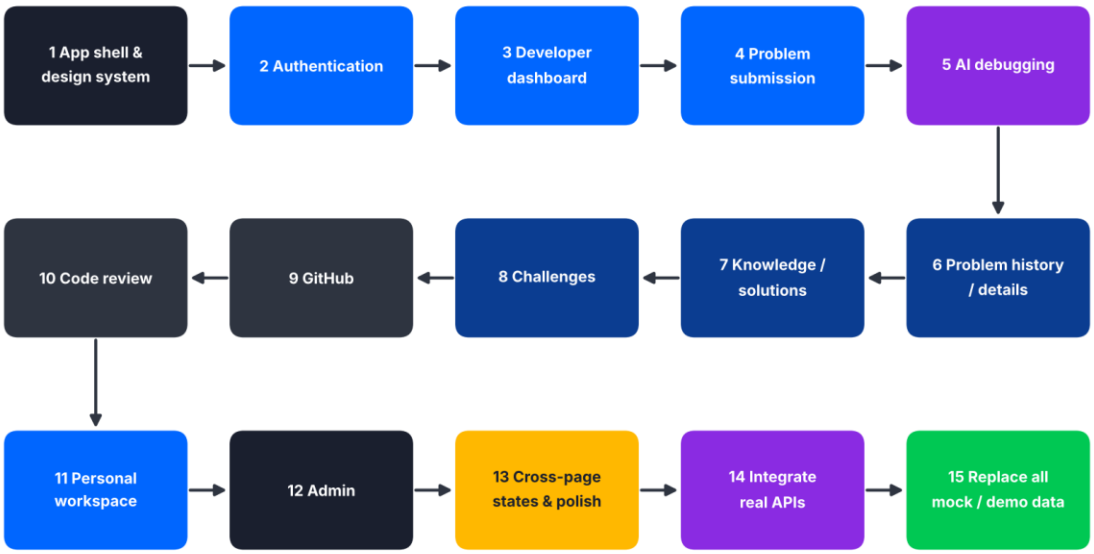


Figure 25 – Recommended build order

Build the shell first

- Theme
- Header
- Sidebar
- Footer
- Typography
- Buttons
- Inputs
- Cards
- Status badges

Recommended Architecture

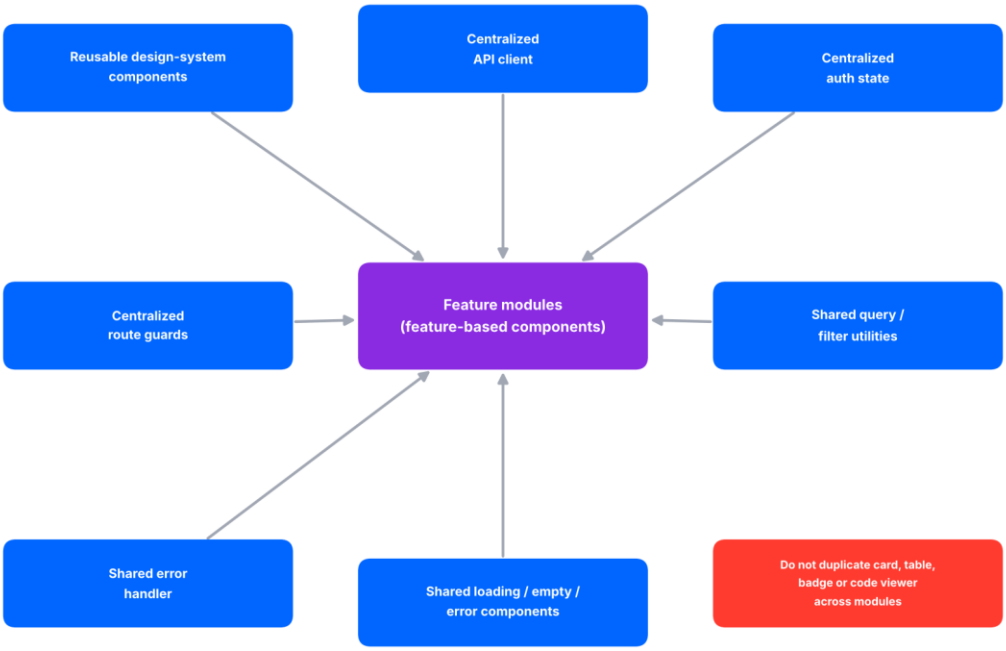


Figure 26 – Recommended frontend architecture

- Reusable design-system components
- Feature-based components
- Centralized API client
- Centralized auth state
- Centralized route guards
- Shared query/filter utilities
- Shared error handler
- Shared loading/empty/error components

> **Rule**
> Do not duplicate the same card, table, badge, or code viewer in multiple modules.

29. Final Business Acceptance Checklist

The frontend is aligned only when all of the following are true:

☐	Developer can register.
☐	Developer must verify email.
☐	Developer can log in with email/password.
☐	Developer can submit a real coding problem.
☐	Developer can provide code/errors/logs/screenshots.
☐	AI debugging shows root cause + steps + why each step works.
☐	Developer can select Solved.
☐	Developer can select Still Have a Problem.
☐	Unresolved problems can continue through multiple attempts.
☐	Previous debugging attempts are preserved.
☐	Successful solutions are stored.
☐	Similar knowledge can be reused.
☐	Public users can browse knowledge articles.
☐	Article pages show complete educational structure.
☐	Start Challenge is available only to authenticated developers.
☐	Challenges can be started and evaluated.
☐	GitHub can be connected.
☐	Repositories can be browsed according to permissions.
☐	Repository debugging uses relevant context only.
☐	AI code review can be requested.
☐	Review findings are displayed clearly.
☐	Activity reflects real platform events.
☐	Bookmarks support knowledge reuse.
☐	Notifications reflect actual supported events.
☐	Settings contain only supported capabilities.

☐	Admin routes are protected.
☐	No provider secrets are exposed.
☐	No Brevo secrets are exposed.
☐	No GitHub client secrets are exposed.
☐	No LLM provider keys are exposed.
☐	No fake metrics are used in production.
☐	UI remains consistent with approved DevPilot AI branding.

30. Final Frontend Principle

> **Principle**
> The frontend is NOT a collection of independent pretty pages. Every page must contribute to one connected developer journey.

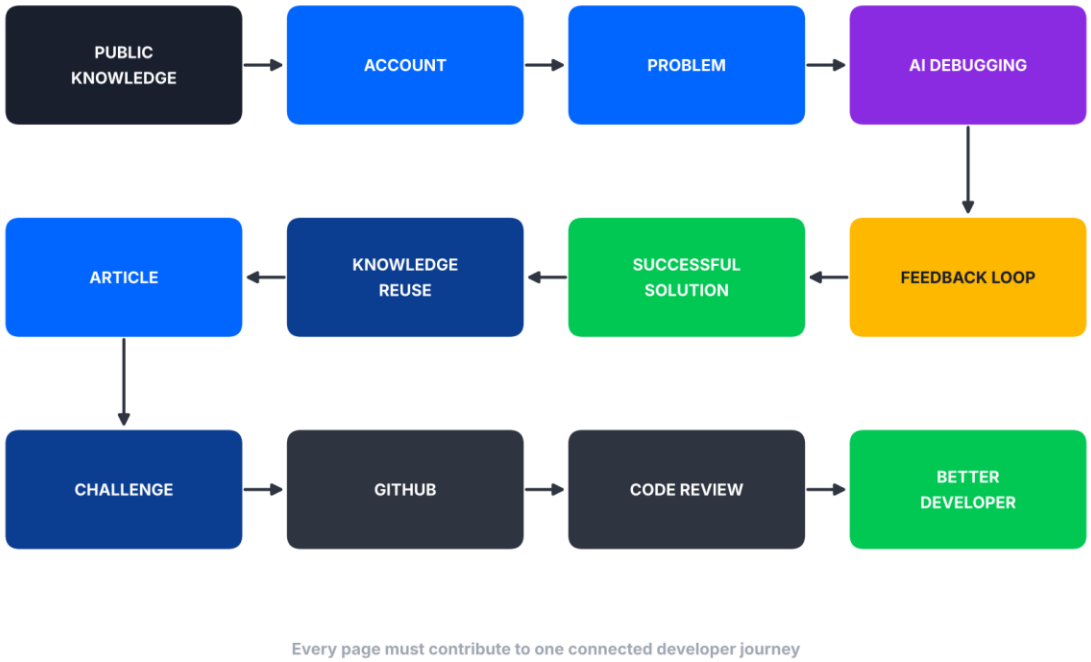


Figure 27 – The connected developer journey

The UI should make this journey clear, fast, professional, and consistent.